Linear Classification

Fatemeh Mansoori

Classification

- Linear Classifier
  - Perceptron
- Probabilistic
  - Naive bayse
- Linear and probabilistic
  - Logistic regression
- Non parametric
  - KNN
  - DT
- Ensemble
  - Random Forest
  - AdaBoost

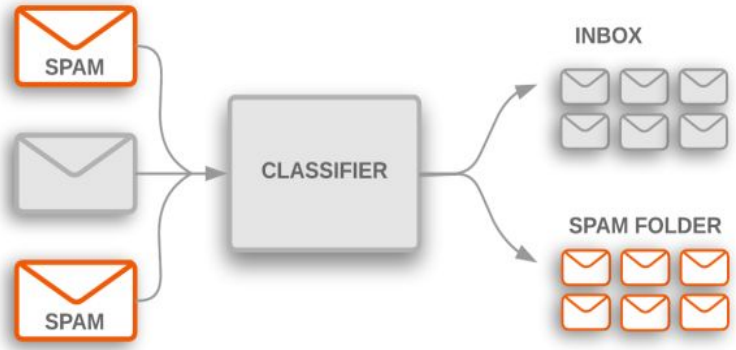
Given: Training Set

- A dataset D with N labeled instances $D = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$
- $y^{(i)} \in \{1, \ldots, K\}$

Goal: Given an input x, assign it to one of K classes.

Real-World Examples:

- Email Spam Detection
- Medical Diagnosis
- Churn Prediction



Real-World Example of Classification

- **Pima Indians Diabetes Dataset:**
  - **Problem**: Predict whether a patient has diabetes based on medical diagnostics.
  - **Context**: Early detection of diabetes is critical for treatment and management.

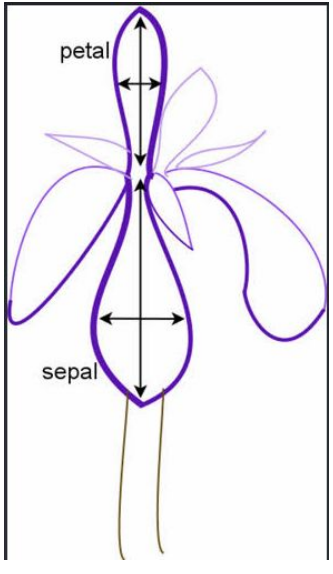
	Number of times pregnant	Glucose	Blood Pressure	Skin Thickness	Insulin	Diabetes pedigree function	Age	BMI	Label
Patient 1	6	148	72	35	0	0.627	50	33.6	Positive
Patient 2	1	85	66	29	0	0.351	31	26.6	Negative
Patient 3	0	137	40	35	168	2.288	33	43.1	Positive
Patient 4	1	89	66	23	94	0.167	21	28.1	Negative
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.

Iris dataset

Sample	Sepal length (cm)	Sepal width (cm)	Petal length (cm)	Petal width (cm)	Species
Sample 1	5.1	3.5	1.4	0.2	Setosa
Sample 2	7.0	3.2	4.7	1.4	Versicolor
Sample 3	6.3	3.3	6.0	2.5	Virginica
Sample 4	4.9	3.0	1.4	0.2	Setosa



Virginica Versicolor Setosa



Classification vs Regression

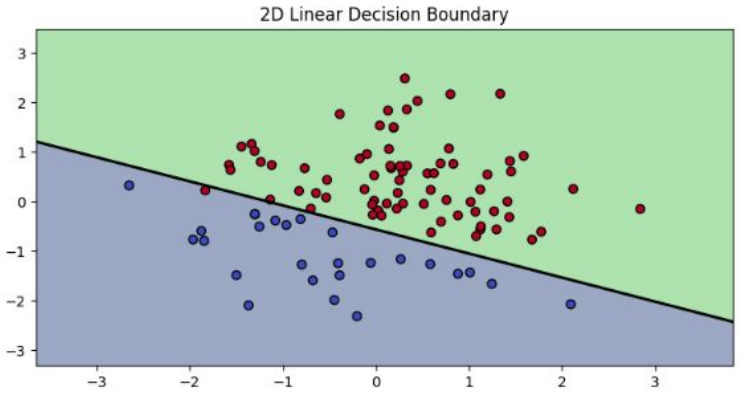
Aspect	Linear Regression	Linear Classification
Output Type	Continuous values (real numbers).	Binary or Multi-class labels (e.g., -1/+1, A/B/C)
Use Cases	Predicting house prices, stock market trends.	Email spam detection, Credit Scoring, Churn Prediction

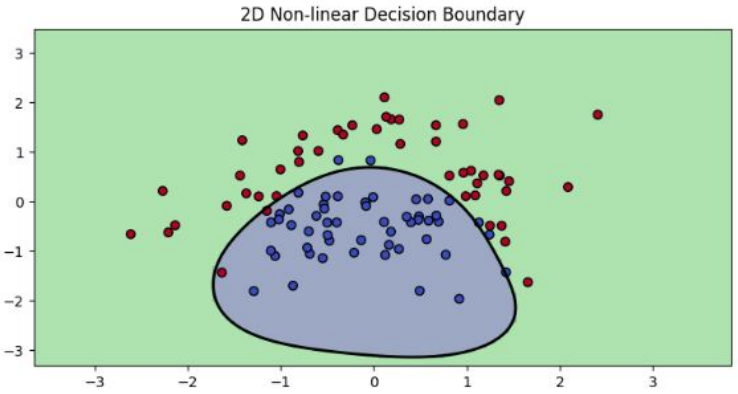
Discriminant Functions in Machine Learning

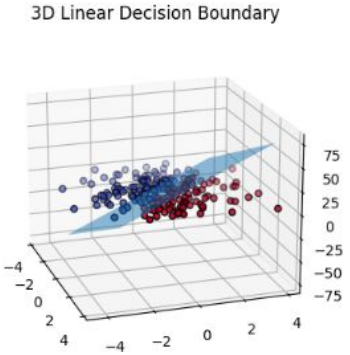
Definition

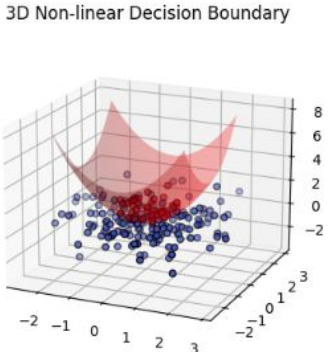
- A function that assigns a score to an input vector x, to classify it into different classes.
- It maps the input $\mathbf{x}$ to a real number $g(\mathbf{x})$, which represents the degree of confidence in assigning $\mathbf{x}$ to a particular class.

Decision Boundary









Linear Classifiers

- **Definition**: In case of linear classifiers, decision boundaries are linear in d ($\mathbf{x} \in \mathbb{R}^d$), or linear in some given set of functions of x.

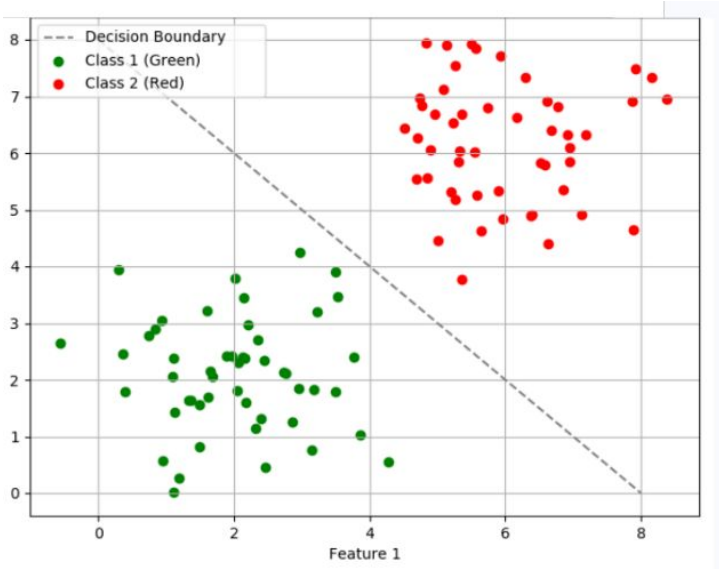
- **Linearly separable data**: Data points that can be exactly separated by a linear decision boundary.

- **Why are they popular?**
  - Simplicity, Efficiency, Effectiveness.

Two Category Classification

$$f(\boldsymbol{x};\boldsymbol{w}) = \boldsymbol{w}^T\boldsymbol{x} + w_0 = w_0 + w_1x_1 + \ldots w_dx_d$$

- $\boldsymbol{x} = [x_1\ x_2\ \ldots x_d]$
- $\boldsymbol{w} = [w_1\ w_2\ \ldots w_d]$
- w_0: bias



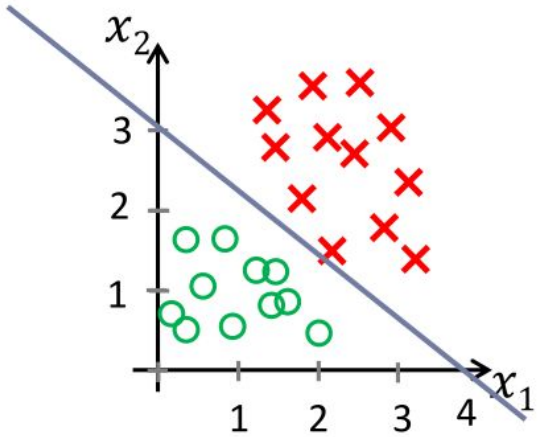
if $\boldsymbol{w}^T\boldsymbol{x} + w_0 \geq 0$ then $\mathcal{C}_1$

else $\mathcal{C}_2$

Decision surface (boundary): $\boldsymbol{w}^T\boldsymbol{x} + w_0 = 0$

$\boldsymbol{w}$ is orthogonal to every vector lying within the decision surface

$$3 - \frac{3}{4}x_1 - x_2 = 0$$

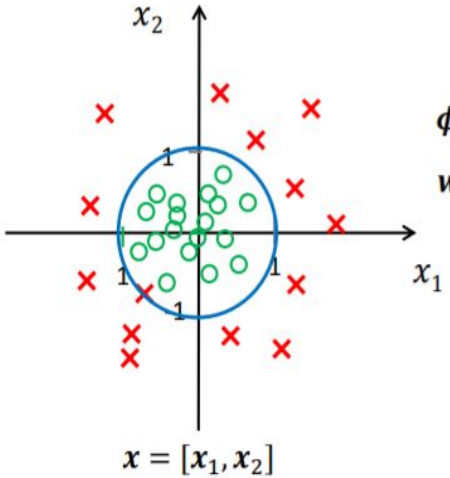


if $\boldsymbol{w}^T\boldsymbol{x} + w_0 \geq 0$ then $\mathcal{C}_1$
else $\mathcal{C}_2$

Non-linear decision boundary

Non-linear Decision Boundaries

- **Feature Transformation**: Non-linearity is introduced by transforming features into a higher-dimensional space.
- **Linear in Transformed Space**: The decision boundary becomes linear in the new space, but non-linear in the original space.



$$-1 + x_1^2 + x_2^2 = 0$$

$$\boldsymbol{\phi}(\boldsymbol{x}) = [1, \boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_1^2, \boldsymbol{x}_2^2, \boldsymbol{x}_1\boldsymbol{x}_2]$$

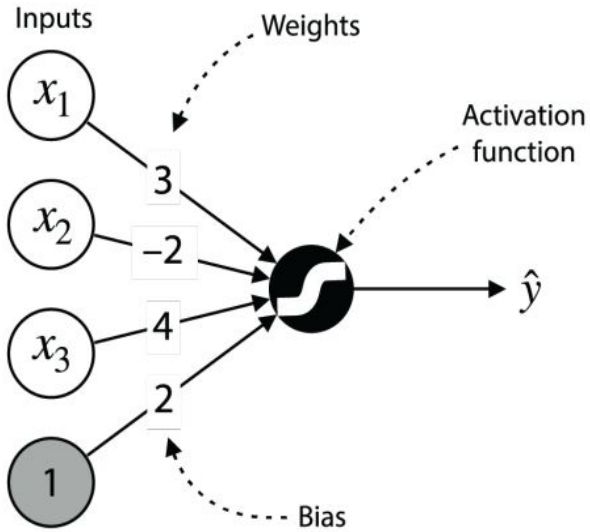
$$\boldsymbol{w} = [w_0, w_1, \ldots, w_m] = [-1, 0, 0, 1, 1, 0]$$

if $\boldsymbol{w}^T\boldsymbol{\phi}(\boldsymbol{x}) \geq 0$ then $y = 1$
else $y = -1$

What Is Perceptron?

Perceptron Unit:

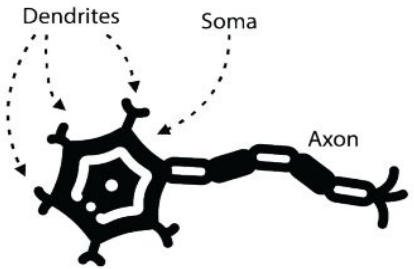
- **Basic Building Block**: A perceptron is the simplest type of artificial neuron used in machine learning.
- **Linear Classifier**: It maps input features to an output by applying a linear combination and a threshold.
- **Binary Decision**: Outputs 1 if the weighted sum of inputs exceeds the threshold, otherwise 0.
- **Components**: Inputs, weights, bias, and an activation function (often a step or a sigmoid function).



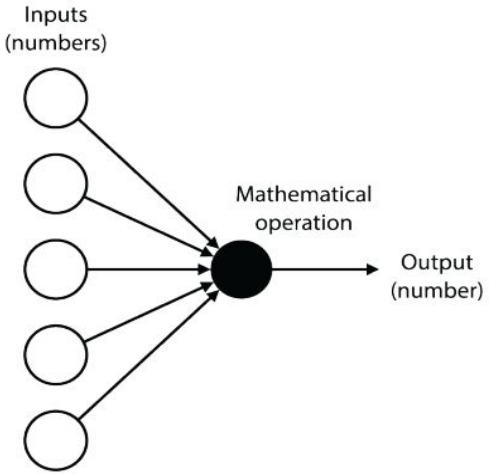
Inspired by Biology

Biological Motivation Behind Perceptron:

- **Inspired by Neurons**: Perceptron mimics the basic function of biological neurons in the brain.
  - Input and Output, Activation Function.



Neuron



Perceptron

Single Neuron

Single Neuron as a Linear Decision Boundary

- **Mathematical Form**: The output of a single neuron is computed as:

$$y = f(\mathbf{w}^T\mathbf{x} + w_0)$$

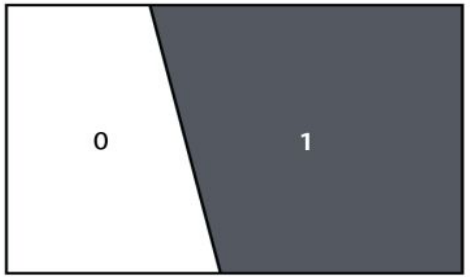
where:

  - $\mathbf{x}$ is the input vector.
  - $\mathbf{w}$ is the weight vector.
  - w_0 is the bias term.
  - f is an activation function (e.g., step function).

- **Linear Separation**: A neuron defines a linear decision boundary:
$\mathbf{w}^T\mathbf{x} + w_0 = threshold$ (0 for step, 0.5 for sigmoid)

- **Decision Rule**: C_1 if $\mathbf{w}^T\mathbf{x} + w_0 \geq threshold$, otherwise

$$\text{Class} = f(\mathbf{w}^T\mathbf{x} + w_0)$$



Cost Function for linear classification

Finding linear classifiers can be formulated as an optimization problem:

- Select how to measure the prediction loss
  - Based on the training set $D = \left\{\left(\boldsymbol{x}^{(i)}, y^{(i)}\right)\right\}_{i=1}^{n}$, a cost function $J(\boldsymbol{w})$ is defined
- Solve the resulting optimization problem to find parameters:
  - Find optimal $\hat{f}(\boldsymbol{x}) = f(\boldsymbol{x}; \hat{\boldsymbol{w}})$ where $\hat{\boldsymbol{w}} = \underset{\boldsymbol{w}}{\operatorname{argmin}} J(\boldsymbol{w})$

Criterion or cost functions for classification:

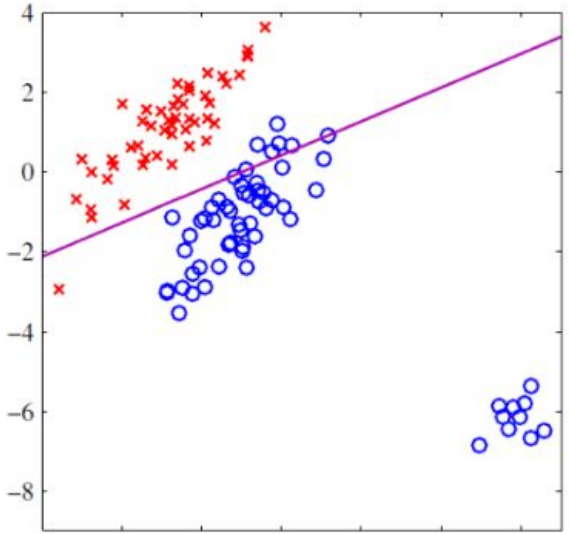
- We will investigate several cost functions for the classification problem

SSE cost function for classification

SSE cost function is not suitable for classification:

- Least square loss penalizes 'too correct' predictions (that they lie a long way on the correct side of the decision)
- Least square loss also lack robustness to noise

Formula: $J(\mathbf{w}) = \sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)})^2, \quad \hat{y}^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} + w_0$



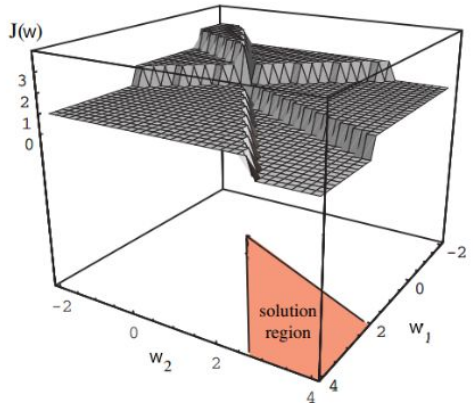
An Alternative for SSE Cost Function

Number of Misclassifications

- **Definition**: Measures how many samples are misclassified by the model.
- **Formula**:

$$J(\mathbf{w}) = \sum_{i=1}^{n} \left(\frac{y^{(i)} - \text{sign}(\hat{y}^{(i)})}{2}\right)^2, \quad \hat{y}^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} + w_0, \quad y^{(i)} \in \{-1, +1\}$$

- **Limitations**:
  - **Piecewise Constant**: The cost function is non-differentiable, so optimization techniques (like gradient descent) cannot be directly applied.



Perceptron criterion

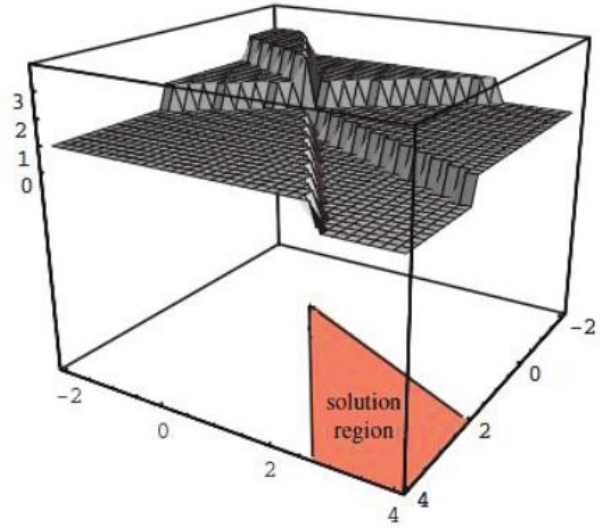
$$J_P(\boldsymbol{w}) = -\sum_{i \in \mathcal{M}} \boldsymbol{w}^T \boldsymbol{x}^{(i)} y^{(i)}$$

$\mathcal{M}$: subset of training data that are misclassified

Many solutions? Which solution among them?

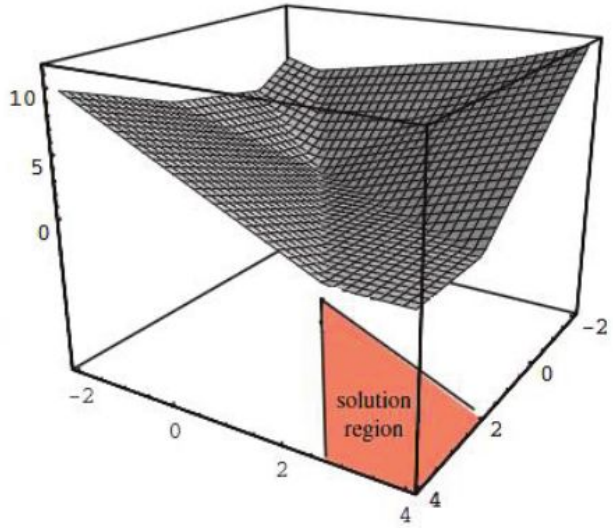
Cost function

$J(\boldsymbol{w})$



of misclassifications
as a cost function

$J_P(\boldsymbol{w})$



Perceptron's
cost function

There may be many solutions in these cost functions

Stochastic gradient descent for Perceptron

Single-sample perceptron:

- If $\boldsymbol{x}^{(i)}$ is misclassified:

$$\boldsymbol{w}^{t+1} = \boldsymbol{w}^{t} + \eta \boldsymbol{x}^{(i)} y^{(i)}$$

Perceptron convergence theorem: for linearly separable data

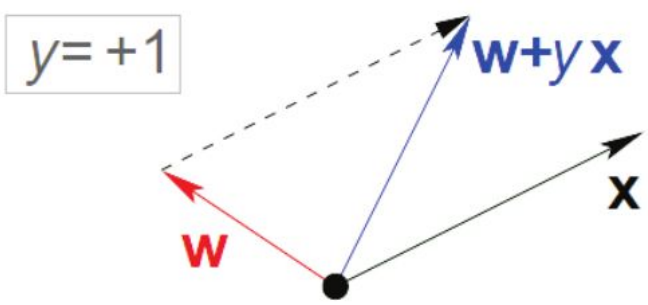
- If training data are linearly separable, the single-sample perceptron is also guaranteed to find a solution in a finite number of steps

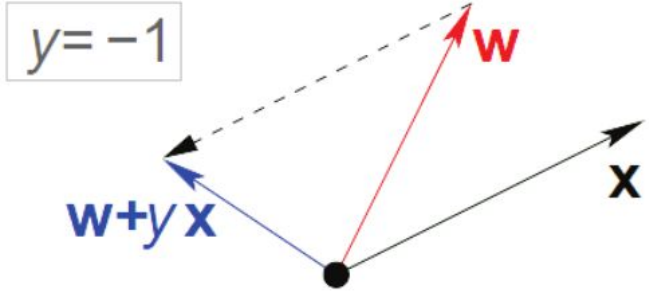
η can be set to 1 and proof still works →

Fixed-Increment single sample Perceptron

Initialize $\boldsymbol{w}, t \leftarrow 0$
repeat
  $t \leftarrow t + 1$
  $i \leftarrow t \bmod N$
  if $\boldsymbol{x}^{(i)}$ is misclassified then
    $\boldsymbol{w} = \boldsymbol{w} + \boldsymbol{x}^{(i)} y^{(i)}$
Until all patterns properly classified

Example





Pocket algorithm

For the data that are not linearly separable due to noise:

- Keeps in its pocket the best $\boldsymbol{w}$ encountered up to now.

Initialize $\boldsymbol{w}$
for $t = 1, \ldots, T$
    $i \leftarrow t \bmod N$
    if $\boldsymbol{x}^{(i)}$ is misclassified then
        $\boldsymbol{w}^{new} = \boldsymbol{w} + \boldsymbol{x}^{(i)} y^{(i)}$
        if $E_{train}(\boldsymbol{w}^{new}) < E_{train}(\boldsymbol{w})$ then
            $\boldsymbol{w} = \boldsymbol{w}^{new}$
end

$$E_{train}(\boldsymbol{w}) = \frac{1}{N} \sum_{n=1}^{N} \left[sign(\boldsymbol{w}^T \boldsymbol{x}^{(n)}) \neq y^{(n)} \right]$$

Multi-class classification: linear machine

A discriminant function $f_i(\boldsymbol{x}) = \boldsymbol{w}_i^T \boldsymbol{x} + w_{i0}$ for each class $\mathcal{C}_i$ $(i = 1, \ldots, K)$:

- $\boldsymbol{x}$ is assigned to class $\mathcal{C}_i$ if:

$$f_i(\boldsymbol{x}) > f_j(\boldsymbol{x}) \;\; \forall j \neq i$$

References

- M. Soleimani, *Machine Learning Course Lecture Slides*.
- A. Sharifi zarchi, Machine Learning Course Lecture Slides
- C. Bishop, Pattern Recognition and Machine Learning, Chapter 4.1.7
- E. Alpaydin, *Introduction to Machine Learning* (4th ed.), chapter 10.